

Supervisor Meeting Prep - Q&A Study Guide (Plain-English Version)

This is written so you can understand and explain every part of your own project, even if you are not technical. Every "Short answer" is a one or two sentence version you can say out loud without needing to understand the deeper mechanics. The "In more detail" part underneath is there in case your supervisor asks a follow-up - you do not need to memorise it word for word, just get comfortable with the idea.

All the numbers here are copied from the actual result files in the project (`results/*.csv` , `results/*.json`), not typed from memory, so they are accurate as of today.

If your supervisor asks something not covered here, it is completely fine to say: "I have not run that part yet, it is planned for a later phase" (see Section 11 for what is not done yet). Do not guess or make something up - supervisors notice immediately, and honesty about what's left to do is a normal, expected part of a project like this.

0. Words you'll hear yourself using (read this first)

Think of this like a small dictionary. Skim it once before the meeting.

- **Classifier / model** - a program that learns to guess whether a message is spam or ham.
- **ham** - a normal, real message. **spam** - junk/scam message.
- **Vectorizer** - a tool that turns text into numbers, because a model can only work with numbers, not raw sentences.
 - **BoW (Bag-of-Words)** - just counts how many times each word appears in a message.
 - **TF-IDF** - like BoW, but it turns down the importance of words that appear in almost every message (like "the", "is") and turns up the importance of words that only show up in a few messages, because those are more useful for telling messages apart.
 - **Unigram** = looking at single words. **Bigram** = also looking at pairs of words next to each other. **Trigram** = also looking at three words in a row.
- **Preprocessing** - cleaning the text before feeding it to the model: making everything lowercase, removing punctuation, removing very common filler words (**stopwords**, like

"the", "is", "and"), and **stemming** (cutting a word down to its root, e.g. "winning" becomes "win").

- **Ablation study** - a fancy name for "turn one setting off at a time and see what breaks." You test each cleaning step separately to see if it actually helps or not.
 - **Accuracy** - the percentage of messages the model got right overall. Can be misleading when one type of message (here, spam) is rare.
 - **Precision** - when the model says "this is spam," how often is it actually right.
 - **Recall** - out of all the real spam, how much did the model actually catch.
 - **F1** - one single number that balances precision and recall together.
 - **AUC-ROC** - a score for how good the model is at telling spam and ham apart in general (1.0 = perfect, 0.5 = no better than a coin flip).
 - **FPR (False Positive Rate)** - the percentage of real, legitimate messages that got wrongly flagged as spam. **This is the number your supervisor cares about most.**
 - **c (a setting for two of the models)** - roughly, how strict vs. flexible the model's decision rule is allowed to be. Think of it as a dial between "keep the rule simple and safe" and "let the rule bend more to fit the training examples exactly."
 - **alpha (a setting for the Naive Bayes model)** - a "smoothing" dial that stops the model from being 100% certain about a word it has barely seen before.
 - **Cross-validation (CV)** - instead of testing once, you split your training data into several chunks, train on most of them and test on the leftover chunk, and repeat this several times with a different leftover chunk each time. It's like practicing with several different mock exams instead of just one, so your estimate of how good the model is becomes more trustworthy.
 - **McNemar's test / paired t-test** - statistical tests that check whether one model being numerically better than another is a real, meaningful difference, or could just be luck.
 - **SHAP** - a tool that explains, for one specific message, exactly which words pushed the model toward calling it spam or ham.
-

1. What is this project, in one paragraph

Title: Comparative Analysis of Classical Machine Learning Classifiers for Spam Detection.

It's a research-style project, not an app - the output is Jupyter notebooks (code + results together), a written report (~6,000 words), and a presentation. You are comparing a handful of well-established, "classical" (not deep-learning) machine learning methods to see which one best tells spam text messages apart from real ones.

Short answer - what got trimmed and why: Your supervisor asked you to cut the project down to something you could finish to a high standard, rather than attempt something bigger and risk finishing it poorly.

In more detail, what changed:

- Dropped one model (Random Forest). Kept four: Naive Bayes, Logistic Regression, Linear SVM, XGBoost.
- Still testing 4 different ways of turning text into numbers (this covers the 3 "strategies" from your proposal, since the n-gram strategy is tested at two levels: pairs of words and triples of words). That's 4 models x 4 text representations = 16 total combinations tried.
- The "which cleaning steps actually help" experiment only runs on the best 2 models from the main comparison, not all 4, and only tests a handful of cleaning combinations, not every possible one.
- The "why did the model decide this" explanation step (SHAP) is limited to the best simple model plus XGBoost, because it is slow to run.
- Everything else from the original plan stayed in: the hybrid features idea, the full set of scoring metrics (including FPR), the two statistical significance tests, and measuring how fast/light each model is.

Q: What's the actual question this project is trying to answer?

A (short answer): Which model and which way of turning text into numbers catches spam best, while keeping real messages from being wrongly flagged - and does cleaning the text less aggressively actually work better than cleaning it a lot?

2. The dataset (the SMS messages you're working with)

Where it's from: A well-known public dataset called the UCI/Kaggle SMS Spam Collection (`data/spam.csv`). Each row has a label (ham or spam) and the message text.

Q: How many messages do you actually have?

A (short answer): Started with 5,572 messages, ended up with 5,158 after removing exact duplicates.

In more detail: 414 messages (about 7.4%) were exact repeats of another message, so they were removed before anything else happened. This matters because if the same message appeared twice, one copy could end up used for training and the other for testing, which would let the model "cheat" by having already seen that exact message.

Q: How many are spam vs. real messages?

A: 4,516 are ham (real), 642 are spam - so spam makes up about 12.45% of the data. That's an uneven ("imbalanced") split, and it shapes several other decisions in the project (see Section 4).

Q: Did you find any problems in the raw data?

A (short answer): Yes - about 50 messages had a comma inside the message text itself, which confused the file format and could have silently chopped off part of the message.

In more detail: the dataset is a CSV file, which normally separates columns using commas. If a message itself contains a comma (e.g. "Call me, ok?"), a simple reader would think that comma starts a *new column*, and the rest of the message would spill into extra, mostly-empty columns instead of staying in the main text column. A careless script that only reads the main column would then silently cut that message short - no error, no warning, just quietly wrong data. Your loading code checks for this and stitches the message back together correctly. You double-checked that every affected message comes back whole.

Q: Is there anything obviously different about spam messages vs. real ones?

A: Yes - spam messages are noticeably longer on average (about 138 characters vs. 71 for real messages), and use more words (about 24 vs. 14). Spam text tends to run close to the old 160-character SMS length limit, since it's usually a promotional or scam message trying to fit in a phone number, a prize claim, or a link.

Q: Why remove duplicate messages before splitting the data into "train" and "test" groups, rather than after?

A (short answer): Removing duplicates first guarantees no message can appear in both groups at once; removing them after does not guarantee that.

In more detail: imagine the same message appears twice in the raw data. If you split first, one copy might land in the "training" pile and the other in the "testing" pile. Then, if you only remove duplicates separately *within* each pile afterward, that cross-pile duplicate never gets caught - it's not a duplicate *within* either pile on its own. The model would then be tested on a message it already memorised while training, making it look better than it really is. Removing duplicates from the *whole* dataset before any split happens means each message can only exist once, so it can only ever end up in one pile.

Q: What words show up most in spam vs. ham?

A: Spam messages are full of promotional/scam words - "free", "call", "txt", "claim", "mobile", plus various numbers and prize-related words. Real messages use ordinary, conversational words. This lines up with a later finding (Section 9) that punctuation like

"!" and "£" also helps spot spam - spam has its own recognisable style, not just its own vocabulary.

3. Making sure results can be repeated (this is something you're specifically graded on)

Q: How do you make sure someone else could re-run this and get the same answers?

A (short answer): Four things: one fixed "random seed" used everywhere, one saved train/test split reused by every notebook, every reported number traced back to a saved file (nothing typed from memory), and exact software versions locked in place.

In more detail, one at a time:

1. **A fixed random seed (the number 42).** Several steps in this project involve some randomness (like shuffling data before splitting it). Using the same "seed" number every time means that randomness produces the exact same result every time it's re-run, instead of a different random outcome each time. It's set in one place (`src/config.py`) and reused everywhere, never re-typed elsewhere.
2. **One saved 80/20 split.** Right at the start, the data was split once into 80% for training and 20% for testing, and that exact split was saved to a file. Every notebook after that reloads the same saved split instead of creating a new random one, so every experiment is judged against the exact same held-back test messages.
3. **No hand-typed numbers.** Every number quoted in the report has to come from a results file the code actually produced - never copied from memory or typed by hand. If a number needs quoting, the file gets re-read.
4. **Locked software versions.** The exact version of every tool used (pandas, scikit-learn, etc.) is written down in `requirements.txt` , so someone else's computer installs the exact same versions, avoiding "it works on my machine" problems.

Q: You mentioned the split is "stratified" - what does that mean and why bother?

A (short answer): It means both the training pile and the testing pile keep roughly the same percentage of spam as the whole dataset (~12.45%), instead of ending up unevenly by chance.

In more detail: without this, a plain random split could, just by bad luck, put very few spam messages into the test set, making your test results shaky and unreliable. You checked this worked: training data ended up 12.46% spam, testing data 12.40% spam - both very close to the overall 12.45%.

Q: What is cross-validation and why did you use it?

A (short answer): Instead of tuning the model on one training pass, you tested it 5 different ways on different slices of the training data and averaged the result, so the tuning process isn't relying on one lucky (or unlucky) split.

In more detail: it's "stratified 5-fold" cross-validation, meaning each of the 5 slices also keeps roughly the same spam percentage. Same fixed seed as everything else, so it's repeatable.

Q: You use TF-IDF, which has to "learn" from the data first. How do you stop that from accidentally peeking at the test data?

A (short answer): The text-to-numbers step and the model are bundled together as one unit, and that whole unit is only ever "taught" using training data - it never sees the test data while learning.

In more detail: in code terms this is a scikit-learn `Pipeline`, and it's important because otherwise the vectorizer could learn, say, "this exact test message's wording exists," which would be an unfair advantage - a form of what's called data leakage.

4. Dealing with spam being rare (only ~12% of the data)

Q: The dataset is mostly real messages - how did you stop the model from just ignoring spam?

A (short answer): Two things - told the models to pay extra attention to the rare spam class while learning, and judged performance using metrics that don't get fooled by imbalance.

In more detail:

- For two of the models (Logistic Regression, Linear SVM), there's a setting (`class_weight="balanced"`) that tells the model "getting a spam message wrong costs more than getting a ham message wrong," roughly in proportion to how rare spam is. For XGBoost, there's a similar setting (`scale_pos_weight`) computed from the training data only.
- You never judge success by plain accuracy alone, because a model that just guessed "ham" every single time would already be about 87.5% accurate while catching zero spam - accuracy alone would hide a completely useless model. Instead you look at the full set of metrics (precision, recall, F1, AUC, and FPR), and use F1 as the main number for picking the best settings, because it balances catching spam against not crying wolf on real messages.

Q: One of your models, Naive Bayes, doesn't have that "pay extra attention" setting. Does that make it unfair?

A (short answer): Naive Bayes was left at its normal, un-adjusted setting for this - it's

worth naming as a known gap, but you can't say for sure that's why it scored lower.

In more detail: Naive Bayes has a smoothing setting (`alpha`) but nothing that directly corrects for spam being rare. That said, be careful not to overclaim this as the reason it did worse - in your own results, Naive Bayes (F1 0.9317) actually beat XGBoost (F1 0.9274), and XGBoost *does* have an imbalance-correcting setting. So imbalance handling alone doesn't explain the model rankings - the more likely explanation is the one in Section 8, about how well each type of model suits this kind of text data.

5. The three ways text gets turned into numbers

There are 4 versions used, covering 3 underlying ideas:

Name	What it does
BoW	just counts word occurrences
TF-IDF (unigram)	counts words, but down-weights extremely common ones
TF-IDF (bigram)	same idea, but also looks at pairs of words together
TF-IDF (trigram)	same idea, but also looks at three words together

Q: In your own words, what's the difference between Bag-of-Words and TF-IDF?

A (short answer): BoW just counts words. TF-IDF also cares about how *unusual* a word is, giving more weight to words that stand out.

In more detail: imagine every message uses the word "the" - that word tells you nothing about whether it's spam. TF-IDF automatically turns the importance of super-common words down, and turns the importance of rarer, more distinctive words up, because those distinctive words are actually useful for telling spam and ham apart. In your results, TF-IDF beat plain BoW.

Q: Your own notes say adding word-pairs/triples (n-grams) barely helps. Why still test them?

A (short answer): Because the original approved plan specifically asked for it, and "it didn't help" is itself a useful, honest finding to report - not a mistake to hide.

In more detail: your supervisor's scope trim removed some *models/testing depth*, but kept the requirement to test text representations at two n-gram levels. Reporting "we tried adding more complexity and it didn't improve things" is a legitimate scientific result: it shows the simple, single-word version was already doing the important work.

6. The four models being compared

Model	Plain description	What was tuned
Naive Bayes	a simple, fast, probability-based classic spam-filter method	a smoothing amount
Logistic Regression	draws a straight-line-style boundary between spam and ham	how strict the boundary is
Linear SVM	similar idea to Logistic Regression, tends to do well on text	how strict the boundary is
XGBoost	builds lots of small decision trees and combines their votes	tree count, depth, learning speed

Q: Why these four models specifically?

A (short answer): They represent four genuinely different approaches, so comparing them tells you something meaningful, not just four near-identical options.

In more detail: Naive Bayes is the traditional, simple spam-filter method. Logistic Regression and Linear SVM both draw a fairly simple, straight-line-style boundary between the two classes, and are known to work well on text data. XGBoost is a much more complex, tree-based method, included specifically to see whether "more complex" actually means "better" here (it turns out it doesn't - see Section 8).

Q: You tuned every model to get the best F1 score. But your supervisor cares most about FPR. Isn't that a mismatch?

A (short answer): Yes, genuinely - and it's worth bringing up yourself rather than waiting to be asked. Picking by F1 does not always pick the same option as picking by FPR.

In more detail, with real numbers: Linear SVM using single words scores F1 0.9482 with 4 false alarms. Linear SVM using word-pairs scores F1 0.9478 (barely different) but only 3 false alarms. F1 alone would pick the single-word version, but if the goal is fewest false alarms, the word-pairs version is actually slightly better. The best *model* doesn't change either way (it's Linear SVM both times) - only the choice of *which text-representation setting* to report changes depending on which metric wins.

Q: What tuning settings actually won?

A: Linear SVM ended up preferring a "moderate strictness" setting across the TF-IDF versions. Logistic Regression consistently preferred a "let it fit closely" setting. Naive Bayes wanted

less smoothing as the word-combination settings got richer. XGBoost didn't settle on one consistent winning setting across the different text-representation versions - which itself is a small hint that XGBoost wasn't finding as stable/reliable a fit as the simpler models.

7. Why "false positive rate" matters most here

Q: What is FPR, and why is it the main thing your supervisor watches?

A (short answer): FPR is the percentage of real, legitimate messages that got wrongly caught as spam. Your supervisor prioritises it because losing a real message is worse than letting one spam message through.

In more detail: think of it like a smoke alarm. A false alarm (going off when there's no fire) is annoying and erodes trust; missing a real fire is far worse in an absolute sense, but a spam filter that keeps blocking your real messages quickly becomes unusable and gets turned off or ignored. That's the reasoning behind treating a wrongly-blocked real message as the costlier mistake here. It's also why you never rely on plain accuracy alone - accuracy would look great even for a filter that catches zero spam, simply because most messages are ham anyway.

Q: Can you explain the four numbers - true positive, false positive, etc - simply?

A: Think of "positive" as "the model said: this is spam."

- **True positive** - it said spam, and it really was spam. Correct catch.
 - **True negative** - it said ham, and it really was ham. Correctly left alone.
 - **False positive** - it said spam, but it was actually a real message. A wrongful block - the bad outcome FPR is counting.
 - **False negative** - it said ham, but it was actually spam. A miss.
-

8. Main results - comparing all 16 combinations

Model	Text representation	F1 score	False-alarm rate (FPR)
Linear SVM	single words	0.9482 (best F1)	0.0044
Linear SVM	word pairs	0.9478	0.0033 (best FPR)
Linear SVM	word triples	0.9478	0.0033

Model	Text representation	F1 score	False-alarm rate (FPR)
Logistic Regression	word pairs	0.9444	0.0055
Naive Bayes	plain word counts	0.9317	0.0055
XGBoost	plain word counts	0.9274	0.0055

Q: What's the headline result?

A (short answer): Linear SVM was the best model overall, Logistic Regression a close second, and the simpler models both clearly beat XGBoost, the most complex model in the comparison. In more detail: this is actually a genuinely interesting finding, because XGBoost is generally considered a more powerful, modern technique, and it still came out weakest here. XGBoost's score even got *worse* the more word-combinations it was given (0.914 with single words, down to 0.888 with triples), which suggests it was starting to latch onto meaningless noise rather than genuine spam patterns as things got more complex.

Q: Why would the "smarter" model, XGBoost, do worse than simpler ones?

A (short answer): Text data turned into numbers ends up as a huge number of mostly-empty columns, and the simpler, straight-line-style models happen to suit that kind of data naturally, while a tree-based model like XGBoost struggles more with it.

In more detail: imagine a spreadsheet with thousands of columns, one per word, where almost every cell is zero because any one message only uses a small handful of those words. A straight-line-style rule (like "add up points for these specific spammy words") works naturally in that kind of space. A tree-based model has to build a sequence of "if this, then that" branching questions over the same huge, sparse space, which is a harder problem to solve well with a training set this size - and it appears to start reacting to random noise instead of real patterns as the space grows (more n-grams = more noise to get distracted by).

Q: Which text representation worked best?

A: TF-IDF (weighted counts) beat plain Bag-of-Words counts. Adding word-pairs or word-triples on top of single words made almost no difference for the best models, and actually made XGBoost worse. So "single words with TF-IDF weighting" is close to the sweet spot - and between word-pairs and word-triples specifically, since they scored identically, the simpler word-pairs option wins because there's no reason to pay the extra complexity cost for zero benefit.

Q: Did you hit any technical problems getting these results?

A (short answer): Yes - one of the models (XGBoost) would freeze/hang forever on your computer the first time you tried to run the full comparison.

In more detail: it turned out to be a conflict between two different pieces of software both trying to use multiple processor threads at the same time, which caused a deadlock (both waiting on each other forever). The fix was to tell one of them to only use a single thread. It didn't change any results, just made the run take a bit longer (about 5 minutes for the full comparison) instead of hanging.

Q: Your test score is actually higher than your tuning (cross-validation) score. Shouldn't a model do worse on data it hasn't seen, not better?

A (short answer): They're not quite the same measurement - the tuning score is an average across several smaller training runs, while the test score comes from one final model trained on all the training data. A gap in either direction is normal, not a red flag.

In more detail: during tuning, each of the 5 cross-validation rounds only trains on about 80% of the training data. The final model used for testing is trained on all of the training data, which is simply more information to learn from. On top of that, the test set only has about 128 spam messages in it, so a handful of messages going one way or the other can shift the score by a percentage point or two just by chance. Neither number is "wrong" - they're measuring slightly different things.

Q: What do your charts/figures actually show?

A, one line each:

- **Class balance chart** - how few spam messages there are compared to real ones (about 12.5% vs 87.5%).
 - **Message length chart** - spam messages are clearly longer on average than real ones.
 - **Top words chart** - the most common words in spam vs. in real messages.
 - **F1 heatmap** - a colour-coded grid showing every model's score at every text representation; darker blue means a higher score, and Linear SVM's TF-IDF cells are the darkest on the whole grid.
 - **FPR heatmap** - same idea, but for the false-alarm rate, your priority metric.
 - **ROC curves** - a chart showing how well each model can separate spam from ham in general; all four models look strong here (they all cluster near the "very good" corner), so the real differences between them mostly show up in the other metrics, not in this chart.
 - **Ablation chart** (Section 9) - a bar chart comparing each text-cleaning setting's score.
-

9. Testing which text-cleaning steps actually help

This part only tests the two best models from Section 8 (Linear SVM, Logistic Regression), on five different cleaning setups, changing one thing at a time from a starting point:
plain cleaning -> plus removing filler words -> plus stemming -> plus both -> and keeping punctuation instead of stripping it out.

Cleaning setup	Linear SVM score	Logistic Regression score
starting point	0.9317	0.9402
+ remove filler words	0.9286 (worse)	0.9291 (worse)
+ stemming	0.9312 (about the same)	0.9444 (better)
+ both together	0.9194 (worst)	0.9170 (worst)
keep punctuation	0.9398 (best)	0.9482 (best)

Q: What's the headline finding here?

A (short answer): Keeping punctuation instead of stripping it out was the single most helpful change, and removing filler words actually hurt performance.

In more detail: characters like "!", "\$", and "£" turn out to carry real spam signal (think of a message like "WIN £1000!!!") - the standard advice to "clean everything" was actually throwing away useful information. Meanwhile, removing so-called "filler" words hurt both models and increased the false-alarm rate too, suggesting those small words aren't just noise in short text messages - they carry some structure that matters. Stemming alone was roughly neutral to slightly helpful. Combining filler-word removal with stemming was the worst setup of all. The overall lesson: less aggressive cleaning worked better here than more aggressive cleaning.

Q: What's this extra "anchor" row I see mentioned in the results file?

A (short answer): It's a sanity check, not a sixth test - it re-runs the exact same setup from the main comparison (Section 8) to prove the numbers still come out identical.

In more detail: the five cleaning tests use a slightly different technical setup than the main comparison, so none of them can be directly compared number-for-number to the Section 8 results. To bridge the two, you added one extra row that repeats the exact Section 8 setup inside this new notebook, and confirmed it reproduces the exact same score down to four decimal places. That confirms the saved data split and everything downstream of it is still working correctly and consistently.

Q: Did you have to fix anything to make the "keep punctuation" test actually work?

A (short answer): Yes - you found and fixed a small bug where punctuation was still being silently deleted even when the setting said to keep it.

In more detail: you proved the fix was safe by running the *original* cleaning setting (the one already used everywhere else in the project) both before and after the fix, across all 5,158 messages, and got exactly zero differences - so nothing that already existed was disturbed, only the new "keep punctuation" option actually started behaving differently, as intended.

Q: How do you know punctuation was genuinely being used by the model afterward, not just kept and then ignored?

A: You checked that the model's vocabulary (the total list of distinct things it recognises) actually got bigger once punctuation was kept - which it did, for both models. If punctuation weren't really reaching the model, the vocabulary size wouldn't have grown.

Q: Could any of this cleaning have accidentally let test messages leak into training?

A (short answer): You checked for it directly rather than assuming it away, and it turned out to be a non-issue.

In more detail: aggressive cleaning can sometimes make two originally-different messages turn into the exact same cleaned text. You measured how often a cleaned test message exactly matched a cleaned training message, for every cleaning setup. It was a small number (7 or 8 messages) for the more aggressive cleaning setups - and interestingly, that same small overlap already existed under the standard cleaning used everywhere else in the project too - and exactly zero for the "keep punctuation" setup, since keeping punctuation makes messages more distinct from each other, not less. Either way, the effect on the scores is negligible.

10. Tools used

- **Python 3.12** specifically - a couple of key tools (XGBoost, SHAP) don't yet fully support the newest Python version, so an older, stable one was chosen deliberately.
- **pandas, numpy, scikit-learn, xgboost, scipy** - the core data/ML toolkit.
- **nltk** - provides the filler-word list and the stemming tool.
- **shap** - planned for later, to explain individual predictions (not run yet).
- **statsmodels** - planned for later, for the statistical significance test (not run yet).
- **matplotlib, seaborn** - used to draw all the charts.
- **jupyter** - the notebook format the whole project is built in; every notebook is fully re-run top to bottom before being saved, so what's on disk really is the last actual run,

not something hand-edited afterward.

- Every tool's exact version is locked in a requirements file, for repeatability.
- The whole project is on GitHub, so you can show your supervisor the code and history directly if useful: <https://github.com/oviroy/sms-spam-classifier-dissertation>.

Q: Why keep the reusable code in a separate folder (`src/`) instead of writing everything directly inside each notebook?

A (short answer): So the same logic - like how the split is made, or how a score is calculated - only exists in one place, and can't quietly drift apart between notebooks.

In more detail: if you copy-pasted the same calculation into five different notebooks and later needed to fix or tweak it, you'd have to remember to fix it in all five places identically, or risk them disagreeing with each other. Keeping it in one shared place means there is exactly one true version of each piece of logic that every notebook uses.

11. What's left to do (be upfront about this in the meeting)

Right now, the finished parts are: environment setup, exploring the data, the main 16-way comparison, and the text-cleaning test. What's still ahead:

- **Combining text features with simple number-based features** - things like message length, how many digits it has, how much punctuation it has - to see if adding those on top of the text-based features helps at all. Not built yet. There's a known small design snag to solve first: one of the models (Naive Bayes) technically breaks if you scale the new number-based features the "normal" way, since it requires all input numbers to stay positive - so a different scaling method (or leaving Naive Bayes out of just that comparison) will be needed.
- **Testing whether the differences are statistically real, explaining the models' decisions, and measuring speed/size** - this is the point where all experiments get "frozen" (no more changes allowed after this, only fixes), right before writing the report properly begins.
- **Writing the actual report and slides** - using everything already done, plus whatever comes out of the two steps above.

Q: If your supervisor asks "what's next?", what's a good simple answer?

A: "Next is combining the text features with a few simple number-based features, like message length. After that comes testing whether my results are statistically meaningful, explaining

why the models make the decisions they do, and measuring how fast and lightweight each model is. Once that's done, I stop changing anything and focus on writing the report."

12. Tough questions your supervisor might ask, and how to answer honestly

Q: Your test set is fairly small (about 1,032 messages, only ~128 of them spam). Can you really trust these results?

A (short answer): It's a fair concern, and you've already partly guarded against it - full certainty is something the still-to-come statistical tests will establish, not something you should claim yet.

In more detail: a small number of spam examples means a handful of mistakes either way can shift the scores noticeably. You reduce this risk somewhat by using cross-validation during tuning (which averages across several training slices, not just one). The planned statistical significance tests (McNemar's test and a paired t-test) are specifically designed to answer "is this difference real, or could it be chance?" - and since those haven't run yet, it's honest to say you haven't made a formal significance claim yet.

Q: You picked the best model using F1, but you said FPR matters most. Isn't that inconsistent?

A: It's a real trade-off, already covered in Section 6/8 - and worth naming yourself rather than waiting for it to be pointed out. The best *model* doesn't change either way, but the best *text-representation setting* does shift slightly depending on which metric you prioritise.

Q: Why not use a more modern approach, like a deep learning / transformer model?

A (short answer): Because the project's whole approved scope is specifically about classical machine learning methods, and for a dataset this size, that's also a sensible, defensible choice on its own merits.

In more detail: transformer-style models are usually overkill for a few thousand short text messages, and are much harder to explain and reproduce cleanly - both things this project cares about. The classical models are already scoring very well here (about 0.948 F1), so there isn't an obvious performance problem that a more complex approach would need to fix.

Q: This dataset is public, and a bit old. Would your results still apply to spam today?

A (short answer): Honestly, not tested, and that's a fair limitation to name rather than avoid. In more detail: spam wording, formatting, and tactics change over time and vary across platforms (SMS vs. email vs. messaging apps), so results on this specific dataset may not

transfer perfectly to today's spam. This would be a good, honest point to mention as "future work" - testing on a newer or different dataset.

Q: Could information have leaked from your test set into training in some way you haven't mentioned?

A: You specifically checked the two most likely ways this could happen: making sure the text-to-numbers step only ever learns from training data, never test data, and making sure no duplicate message could end up on both sides of the split. You also double-checked a subtler version of the second point in the cleaning-test phase - whether *cleaned* text could create new overlaps that didn't exist in the raw text - and measured it directly instead of assuming it wasn't happening.

13. A few more concrete numbers, explained simply

Q: Can you walk me through precision and recall using your actual numbers?

A: For your best setup (Linear SVM, single words): out of 1,032 test messages, the model correctly caught 119 real spam messages, wrongly flagged 4 real messages as spam, correctly left 900 real messages alone, and missed 9 spam messages (let them through as if they were real). That gives:

- **Precision $\approx 96.7\%$** - of everything the model called "spam" ($119 + 4 = 123$ messages), about 97 out of 100 really were spam.
- **Recall $\approx 93.0\%$** - of all the real spam that existed ($119 + 9 = 128$ messages), the model caught about 93 out of 100 of them.

The 9 missed spam messages are effectively the "price" paid for keeping false alarms so low, which lines up with the project's priority on avoiding false alarms.

Q: If you wanted even fewer false alarms, what would happen to the missed-spam number?

A (short answer): They pull against each other - making the model more cautious about calling something spam reduces false alarms, but also lets more real spam slip through, and vice versa. In more detail: this isn't something explored in this project (the models use one fixed, default cutoff point), but it's a natural next step to mention if asked "what would you do for a real product" - you could shift that cutoff point to trade a bit of missed spam for even fewer false alarms, or the other way around, depending on what matters more.

14. If asked about specific technical settings

You don't need to memorise every detail here - these are "if pushed further" answers. The short version for all of them is: *these are standard, sensible default choices for this kind of text data, chosen deliberately rather than left to chance.*

Q: Why these particular internal settings for Logistic Regression and Linear SVM?

A (short answer): Both use settings that are well-suited to text data specifically, which tends to produce a huge number of mostly-empty numeric columns once vectorized. In more detail, if pushed: Logistic Regression uses a solver called `liblinear`, chosen because it handles this kind of data efficiently and works properly with the "pay extra attention to rare spam" setting from Section 4. Linear SVM uses a setting called `dual="auto"`, which just means "let the software automatically pick the more efficient internal method for this data's shape" rather than forcing one choice blindly.

Q: Your settings-search grids are small (only a few options tried per model). Why not try more?

A (short answer): A deliberate, stated trade-off to keep the project's runtime manageable on a normal computer (no special hardware was used).

In more detail: every extra option tried gets multiplied across 5 cross-validation rounds and 4 text-representation versions, so the cost adds up fast - the XGBoost search alone already means 160 separate training runs. The chosen options were picked to sensibly cover "under-tuned" to "over-tuned," not to exhaustively try everything possible.

Q: Why stemming specifically with the "Porter" method, not another one?

A (short answer): It's the standard, most common stemming method used in this kind of work, and it behaves moderately (not too aggressive, not too weak).

In more detail, if pushed: other options exist (Snowball, Lancaster) and would be reasonable too - Porter was a sensible, well-documented default choice, not a claim that it's provably the single best option.

Q: Naive Bayes works differently from the other models under the hood - can you explain that simply?

A (short answer): Naive Bayes tries to learn what "typical spam text" and "typical real text" each look like, then checks which one a new message resembles more. Logistic Regression and Linear SVM instead just learn a direct rule for telling the two apart, without trying to model what each category "looks like" first.

In more detail, if pushed: this distinction (called "generative" vs. "discriminative" in textbooks) is often given as a reason discriminative models tend to do slightly better at pure

classification tasks like this one - which roughly matches what you found, though you shouldn't lean on it too hard as *the* explanation, since XGBoost is also a "direct rule" style model and it still did worst of all four.

15. How this connects to your proposal, plus a couple of likely side questions

Q: How does what you've built so far map back to your original proposal's objectives?

A (short answer): The finished notebooks, the main comparison results, and the cleaning-test results already cover several of your original objectives directly; the remaining objectives are covered by the phases still to come.

In more detail: your project plan (`PLAN.md`) has a table mapping each deliverable to specific numbered objectives from your proposal - the reproducible notebooks and saved split cover the reproducibility objectives, the main comparison results cover the core comparison objectives, and the cleaning-test results cover the ablation objective. The objectives about hybrid features, statistical testing, explainability, and efficiency will be covered once those phases are built.

Q: Is there any data privacy concern, since this is real people's text messages?

A (short answer): No meaningful concern - it's a long-established, publicly shared academic dataset, not new data collected from real people during this project.

In more detail: because it's an existing public research dataset rather than something newly gathered from real users, it doesn't require separate ethics approval, unlike a project that went out and collected fresh SMS data from participants.

Q: When you eventually run the statistical significance tests, what will they actually tell you?

A (short answer): Whether the differences you found between models are likely to be real, or could just be random luck from this particular test set.

In more detail: one test (McNemar's) looks at where two models specifically disagree on the same test messages and checks if that disagreement pattern is meaningful. The other (a paired t-test) instead compares the models across the several cross-validation rounds from earlier, checking if one model is consistently better across different slices of data, not just on one single test.

Q: What will the "explain the model's decisions" step (SHAP) add that you don't already have?

A (short answer): It will show, for one specific message, exactly which words pushed the prediction toward spam or toward ham - which is more useful than a single overall list of "important words" for the whole dataset.

In more detail: right now you can look at a linear model's internal weights to see which words generally push toward spam. SHAP goes further by working at the level of one individual message and, importantly, also works for XGBoost, which doesn't have simple weights to read the same way a linear model does.

16. Quick facts cheat-sheet (for rapid-fire recall)

- Same fixed random setup used everywhere: seed number **42**.
- Data split: **80% training / 20% testing**, done once, saved, and reused everywhere.
Training: 4,126 messages. Testing: 1,032 messages.
- Dataset: started at 5,572 messages, **5,158** after removing 414 exact duplicates.
12.45% of messages are spam.
- Four models compared: **Naive Bayes, Logistic Regression, Linear SVM, XGBoost**.
- Four text-number-conversion methods compared: **plain word counts, TF-IDF single words, TF-IDF word-pairs, TF-IDF word-triples** - 16 combinations total.
- Best result by F1 score: **Linear SVM, single words, F1 = 0.9482, false-alarm rate 0.44%**.
Best result by false-alarm rate specifically: **same model, word-pairs, F1 = 0.9478, false-alarm rate 0.33%** - basically the same score, but one fewer false alarm.
- The metric your supervisor cares about most: **false-alarm rate (FPR)** - wrongly blocking a real message is treated as worse than letting one spam message through.
- Best text-cleaning finding: **keeping punctuation** helped both top models the most.
- Worst text-cleaning finding: **removing filler words AND stemming together** hurt both models the most.
- Done so far: environment setup, data exploration, the 16-way main comparison, the text-cleaning test. Still to come: combining text with simple number features, statistical significance testing, explaining model decisions, measuring speed/size, and writing the report.
- Project repo: <https://github.com/oviroy/sms-spam-classifier-dissertation> .